EI-DDLGN: Efficient Encrypted Inference with Deep Differential Logic Gate Networks under TFHE

No Author Given

No Institute Given

Abstract. The abstract should briefly summarize the contents of the paper in 150–250 words.

Keywords: Privacy-Preserving Inference · Logic Gate Network · Homomorphic Encryption · TFHE.

1 Introduction

The rapid expansion of digital multimedia technologies has led to an unprecedented increase in the generation, transmission, storage, and usage of sensitive data, including medical images, surveillance footage, and personal visual content. Ensuring the confidentiality and integrity of such data has become a fundamental requirement in modern cybersecurity systems. In response, research has been devoted to multimedia protection [19,18] to prevent unauthorized access while maintaining robustness against statistical and cryptoanalytical attacks.

In parallel, the last decade has witnessed remarkable advances in Artificial Intelligence (AI), particularly deep learning, where neural networks have achieved state-of-the-art performance across a wide range of tasks, including image classification, object detection, and medical diagnosis. These models are increasingly deployed in cloud-based environments, where user data is transmitted to remote servers for inference. While this paradigm offers significant computational advantages, it raises serious privacy concerns, as sensitive input data must be revealed to potentially untrusted servers during inference [16].

To address this challenge, privacy-preserving inference of deep learning has emerged as an active area of research, aiming to enable inference without exposing raw data. Fully Homomorphic Encryption (FHE) provides a powerful cryptographic framework for this purpose, as it allows computations to be performed directly on encrypted data without requiring decryption. Among FHE schemes, Torus Fully Homomorphic Encryption (TFHE) has emerged as a powerful approach due to its capability to perform computations of arbitrary depth, eliminating the depth constraints that limit many other homomorphic encryption schemes [4]. This capability is enabled by a key feature of TFHE, namely, programmable bootstrapping (PBS). PBS not only refreshes ciphertext noise but

also enables the evaluation of non-linear functions through lookup tables, making it particularly suitable for privacy-preserving inference of Neural Networks (NNs) [5].

TFHE construction and homomorphic operations are more aligned with Boolean and integer-based plaintext messages. Therefore, Zama proposed to leverage these capabilities to design TFHE-compatible NNs using Quantization-Aware Training (QAT) [17]. In this approach, network weights and activations are quantized during training, allowing inference to be performed entirely over integer representations while preserving accuracy even at very low quantization bit-widths.

However, these advantages come at a high computational cost. The PBS operation is inherently expensive, and its latency grows with the accumulators' bit-width of NN layers, making inference time highly sensitive to quantization choices. Furthermore, the circuit bit-width, determined by the maximum intermediate value across the network, is very sensitive to the chosen initial conditions of the training. This introduces additional variability and complexity in parameter selection, often leading to non-uniform and difficult-to-control performance characteristics. As a result, conventional neural network architectures, even when adapted through quantization, remain inefficient for practical large-scale encrypted inference under TFHE, as will be shown in the upcoming experimental Section 6.

This challenge motivates the exploration of alternative model architectures that preserve the learning capabilities and characteristics of NNs, and at the same time are inherently compatible with TFHE. Deep Differential Logic Gate Networks (DDLGNs) [14] represent a promising solution in this direction. Unlike traditional NNs, DDLGNs are composed of differentiable logic gates that are discretized after training into Boolean circuits. As a result, their inference can be expressed entirely in terms of Boolean operations, making them naturally aligned with the Boolean mode of TFHE. Unlike QAT-based approaches, which rely on integer arithmetic and incur costly PBS operations with circuit bit-width-dependent latency, DDLGNs operate directly on Boolean representations, enabling a more predictable and efficient execution under TFHE.

In this work, we propose Encrypted Inference for Deep Differential Logic Gate Networks (EI-DDLGN) for privacy-preserving inference under TFHE as an alternative framework to the Quantization-Aware Training-Fully Connected Neural Network QAT-FCNN, introduced by ZAMA in [17], in a client-server setting. The client encrypts input data locally and transmits ciphertexts to an untrusted server hosting a trained DDLGN model. The server performs inference directly on encrypted data using Boolean gate evaluations and returns encrypted predictions, which are decrypted by the client. Hence, the proposed framework ensures end-to-end data confidentiality throughout the inference process.

We provide an experimental evaluation of DDLGN models under TFHE across three model sizes. Our results show that encrypted inference latency scales linearly with the total number of Boolean gates, which suggests that gate complexity is the dominant factor governing computational cost. Furthermore, we

show that DDLGNs under TFHE achieve competitive accuracy while significantly outperforming QAT-FCNN in terms of inference latency.

Contributions. The main contributions of this work are as follows:

- We propose EI-DDLGN, the first TFHE-based framework for privacy-preserving inference of DDLGNs.
- We show that Boolean gate complexity in EI-DDLGN is the primary factor determining encrypted inference latency.
- We provide experimental evaluations across three EI-DDLGN model sizes to highlight the practical latency cost and its trade-off with accuracy.
- We compare our proposed framework with the arithmetic-based NN, QAT-FCNN, and show significant improvements in encrypted inference latency and accuracy.

The remainder of this paper is organized as follows: Section 2 reviews the related work. Section 3 introduces the preliminaries on TFHE and DDLGNs. Section 4 presents the proposed EI-DDLGN framework together with its threat model and implementation considerations. Section 5 analyzes the computational complexity and latency cost of EI-DDLGN. Section 6 describes the experimental setup and discusses the obtained results. Finally, Section 7 concludes the paper.

2 Related Work

Privacy-preserving neural network inference under homomorphic encryption has been studied through several architectural and cryptographic design choices. For example, Chillotti *et al.* [5] present one of the key TFHE-based advances for privacy-preserving NN inference by showing how programmable bootstrapping can be used to evaluate deep NNs over encrypted data without requiring model retraining. The paper is built on the TFHE scheme and focuses on using its bootstrapped mode to control noise while simultaneously evaluating functions through lookup-table style computation, thereby overcoming the depth restrictions that limit leveled homomorphic approaches. However, the work still exposes an important limitation of PBS-based TFHE inference. Although it enables arbitrary-depth evaluation, the resulting encrypted inference cost remains substantial, with reported runtimes ranging from seconds to much longer, depending on security level and network depth.

Bourse *et al.* [3] propose FHE–DiNN, a privacy-preserving inference framework for discretized feedforward neural networks under TFHE. Their approach uses the TFHE scheme and refines its bootstrapping procedure so that the *sign* activation function can be applied during bootstrapping itself, while weighted sums are evaluated homomorphically using cleartext weights and encrypted inputs. The paper also focuses on the discretization of NNs by discretizing inputs and weights, and restricting the hidden-layer activation to the sign function. This approach yields a feedforward architecture whose encrypted evaluation becomes linear in the number of neurons and layers rather than globally constrained by

multiplicative depth. The main limitation of this work is the introduction of quantization and discretization loss, which the paper itself notes that directly training DiNNs would be a more principled direction.

A more recent line of work, exemplified by ZAMA's TFHE-compatible NN framework. It uses QAT together with PBS to support arbitrary-depth NNs and general activation functions through lookup tables [17]. This approach is attractive because it preserves exact encrypted computation and turns FHE compatibility largely into a machine learning and quantization problem. However, as we mentioned in the previous section, it still suffers from some practical difficulties, such as PBS remaining expensive, and its cost depends on the circuit bit-width, making accumulator growth, pruning, and quantization choices critical to inference efficiency. This same design philosophy has also been adopted in recent application-driven work on quantized fully connected neural networks for TFHE-based medical image analysis, where QAT and accumulator-aware pruning are used to maintain FHE compatibility while preserving accuracy [16]. In parallel, binary-network-based TFHE inference has also been explored to reduce arithmetic cost, including recent optimizations based on bootstrapping reduction and operational logic [8]; nevertheless, these methods remain rooted in weight-based binary NNs rather than learned logic-gate computation.

In contrast, our work is built on Deep Differentiable Logic Gate Networks (DDLGNs), introduced by Petersen *et al.* [14], where logic gate networks are trained through a differentiable relaxation and then discretized into hard logic-gate networks for fast inference. Unlike binary NNs [8], which remain weight-based and use predefined logical operations to approximate arithmetic computation, logic gate networks learn the logic operations themselves, have no weights, and are intrinsically sparse because each neuron has only two inputs. This distinction is especially important in the encrypted setting: rather than adapting arithmetic NNs to TFHE through quantization and PBS-heavy integer evaluation, our proposed EI-DDLGN offers a more structurally aligned and efficient alternative for privacy-preserving inference under TFHE.

Beyond feedforward encrypted inference, a substantial body of work has also explored privacy-preserving inference for convolutional neural networks (CNNs) under homomorphic encryption. A major example is CryptoNets, which demonstrated that CNN-style encrypted inference is feasible using leveled homomorphic encryption, but relied on polynomial activations and scaled mean pooling to remain compatible with the underlying scheme, thereby introducing architectural approximations and depth-related limitations [7]. Later works, such as Shift-accumulation-based Leveled Homomorphic Encryption (SHE), extended TFHE-based inference to deeper CNNs by implementing ReLU and max-pooling with TFHE Boolean operations and using logarithmic quantization to accelerate convolution and accumulation. That led to achieving strong accuracy improvements over prior CNN methods [12]. More recent efforts further improved CNN inference through architectural and systems-level optimizations, including binary-network-based TFHE inference [2], model redesign for FHEW/TFHE [9], DCT-domain private inference [15], and hybrid encrypted lookup-table frame-

works [11,10]. These works are important contributions to the literature because they show that privacy-preserving encrypted inference can scale beyond FCNN models to richer image-oriented architectures. However, they are outside the direct scope of this paper, since our focus is not on CNNs, but rather on privacy-preserving inference for the feedforward narrative of DDLGNs and their structural compatibility with TFHE.

3 Preliminaries

This section briefly reviews the cryptographic and model background needed for the remainder of the paper. We first summarize the main principles of FHE and TFHE and then discuss the properties of DDLGNs that make them particularly suitable for encrypted inference.

3.1 Fully Homomorphic Encryption and TFHE

FHE enables computations to be performed directly on encrypted data without requiring decryption, thereby providing a strong cryptographic foundation for privacy-preserving deep learning inference [5,3]. In contrast to traditional encryption, where data must be decrypted before processing, FHE allows an untrusted server to evaluate a function over ciphertexts and return an encrypted result that can only be decrypted by the data owner [5]. This property makes FHE an attractive solution where sensitive client data must remain confidential during inference.

A central challenge in FHE is ciphertext noise growth. Homomorphic additions and multiplications increase the noise embedded in ciphertexts, and once that noise exceeds a certain threshold, correct decryption is no longer possible. Leveled homomorphic encryption schemes address this issue only for computations of bounded depth, with parameters chosen in advance according to the target circuit complexity [5,7]. While such approaches have enabled early encrypted neural inference systems, including CryptoNets [7], their efficiency and scalability are strongly constrained by the multiplicative depth, and they often require replacing standard nonlinearities with polynomial approximations such as square activations or scaled mean pooling [12,7].

Among existing FHE families, TFHE has emerged as a particularly relevant framework for private inference because it supports computations of arbitrary depth through bootstrapping and offers efficient operations over Boolean and small-integer representations [5,15]. In TFHE, bootstrapping not only refreshes ciphertext noise but, through PBS, can also evaluate functions during the refresh step itself, typically via lookup-table style. This capability has made TFHE especially attractive for encrypted neural inference, since nonlinear functions can be incorporated directly into the encrypted computation without resorting to polynomial approximation.

At a high level, TFHE encrypts a message by embedding it into a noisy ciphertext, based essentially on the learning with error (LWE) problem for security. Let m denote a plaintext message and let e denote a small error term. A

ciphertext can be viewed as

$$\begin{aligned} &\mathbf{c} \leftarrow \mathrm{LWE}_{\mathbf{s}}(m) = (a_1, a_2, ..., a_n, b) \in \mathbb{Z}_q^{n+1}, \\ &b = \sum_{i=1}^{n} a_i \cdot s_i + e + m, \end{aligned} \tag{1}$$

where n is the LWE dimension, q is the ciphertext modulus, $\mathbf{a} = (a_1, ..., a_n) \xleftarrow{\$} \mathbb{Z}_q$ represents a uniformly random vector and $\mathbf{s} \in \{0,1\}^n$ is the binary secret key. The construction of (1) implies that linear operations between ciphertext experience direct homomorphism with the underlying plaintext values; however, non-linear operations need a different approach. Formally speaking, a complete operation between two ciphertexts $\mathbf{c}_1$ and $\mathbf{c}_2$ can be defined as:

$$\mathbf{c}_r = F(L(\mathbf{c}_1, \mathbf{c}_2)), \tag{2}$$

where $L(.)$ represents the linear operations, e.g. addition and scalar multiplication, and $F(.)$ represents the non-linear function evaluated using a Look-up table (LUT) in the bootstrapping process. As indicated by (1), linear operations between ciphertexts accumulate more noise in the resulting ciphertext, and the decryption process fails if the noise crosses a defined threshold. The bootstrapping process resolves this issue by refreshing the noise content in the ciphertext, represented as:

$$\mathbf{c}_r = \mathtt{R}_{\mathtt{pk},\mathtt{LUT}}\left(L(\mathbf{c_1}, \mathbf{c}_2)\right), \tag{3}$$

where $\mathbf{c}_r$ is the result ciphertext, `pk` is the public key, and `LUT` represents the non-linear function $F(.)$. $\mathbf{c}_r$ experiences two interesting features: its noise content is refreshed to a pre-calculated level that is independent of the noise in $\mathbf{c}_1$ and $\mathbf{c}_2$, and has the function represented by `LUT` evaluated on-the-fly on the result of $L(\mathbf{c_1}, \mathbf{c}_2)$. This property is particularly important for privacy-preserving inference, since nonlinear operations can be evaluated during bootstrapping rather than approximated by low-degree polynomials [5].

3.2 Deep Differential Logic-Gate Networks

DDLGNs are a class of neural architectures in which each neuron is represented by a binary logic gate rather than a weighted arithmetic operator. In the underlying logic-gate network, each neuron receives exactly two inputs, applies one Boolean operator, and forwards the resulting bit to the next layer. As a result, the model is intrinsically sparse, weightless, and purely defined by its logic operations rather than by real-valued weights [14]. This distinguishes DDLGNs from binary neural networks, where logical operations such as XNOR are typically fixed in advance to approximate weight-based arithmetic computation rather than learned directly [8].

Formally, let $x = (x_1, \ldots, x_d) \in \{0,1\}^d$ denote the binary input to a layer, where d is the number of input features (or, more generally, the number of

activations produced by the previous layer). A logic-gate layer is constructed by assigning to each neuron j an ordered pair of input indices (u_j, v_j) with $u_j, v_j \in \{1, \ldots, d\}$, and a Boolean operator

$$f_j : \{0,1\}^2 \to \{0,1\}. \tag{4}$$

The output of neuron j is then given by

$$z_j = f_j(x_{u_j}, x_{v_j}), \tag{5}$$

where x_{u_j} and x_{v_j} are the two selected inputs. In this formulation, the function f_j specifies the logic rule implemented by neuron j, such as AND, OR, XOR, NAND, or any other two-input Boolean operator. Hence, the pair (u_j, v_j) determines which activations are used, while f_j determines how they are combined.

Table 1. All two-input Boolean operators for a logic-gate neuron $z_j = f_j(x_{u_j}, x_{v_j})$, together with their real-valued relaxations, adapted from [14].

ID	Operator	Real-valued relaxation	00	01	10	11
0	False	0	0	0	0	0
1	$x_{u_j} \wedge x_{v_j}$	$x_{u_j} \cdot x_{v_j}$	0	0	0	1
2	$\neg(x_{u_j} \Rightarrow x_{v_j})$	$x_{u_j} - x_{u_j} x_{v_j}$	0	0	1	0
3	x_{u_j}	x_{u_j}	0	0	1	1
4	$\neg(x_{u_j} \Leftarrow x_{v_j})$	$x_{v_j} - x_{u_j} x_{v_j}$	0	1	0	0
5	x_{v_j}	x_{v_j}	0	1	0	1
6	$x_{u_j} \oplus x_{v_j}$	$x_{u_j} + x_{v_j} - 2x_{u_j} x_{v_j}$	0	1	1	0
7	$x_{u_j} \vee x_{v_j}$	$x_{u_j} + x_{v_j} - x_{u_j} x_{v_j}$	0	1	1	1
8	$\neg(x_{u_j} \vee x_{v_j})$	$1 - (x_{u_j} + x_{v_j} - x_{u_j} x_{v_j})$	1	0	1	0
9	$\neg(x_{u_j} \oplus x_{v_j})$	$1 - (x_{u_j} + x_{v_j} - 2x_{u_j} x_{v_j})$	1	0	0	1
10	$\neg x_{v_j}$	$1 - x_{v_j}$	1	1	0	0
11	$x_{u_j} \Leftarrow x_{v_j}$	$1 - x_{v_j} + x_{u_j} x_{v_j}$	1	1	0	1
12	$\neg x_{u_j}$	$1 - x_{u_j}$	1	0	1	1
13	$x_{u_j} \Rightarrow x_{v_j}$	$1 - x_{u_j} + x_{u_j} x_{v_j}$	1	1	0	1
14	$\neg(x_{u_j} \wedge x_{v_j})$	$1 - x_{u_j} x_{v_j}$	1	0	1	1
15	True	1	1	1	1	1

Since a two-input Boolean operator is fully determined by its truth table, the set of admissible functions satisfies

$$|\mathcal{F}| = 2^{2^2} = 16, \tag{6}$$

where $\mathcal{F}$ denotes the family of all mappings from $\{0,1\}^2$ to $\{0,1\}$. Table 1 lists these 16 operators using the notation of neuron j, whose two selected inputs are x_{u_j} and x_{v_j}.

The main difficulty is that such discrete operators are not differentiable, which prevents direct gradient-based training. To address this, Petersen *et al.* [14] propose a differentiable relaxation in which binary activations are replaced

by continuous values in $[0, 1]$, and each neuron learns a probability distribution over the 16 candidate logic gates.

Hence, the two inputs selected by neuron j is relaxed such as $x_{u_j}, x_{v_j} \in [0, 1]$. And let $f^{(i)} \in \mathcal{F}$, $i \in \{0, \ldots, 15\}$, denote the i-th two-input Boolean operator from the admissible family $\mathcal{F}$. In the differentiable relaxation, each hard Boolean operator is replaced by its real-valued counterpart $f^{(i)}(x_{u_j}, x_{v_j})$, so that the output of neuron j becomes continuous rather than binary. For example, common relaxations include AND, XOR, and OR, given respectively by

$$f^{(\wedge)}(x_{u_j}, x_{v_j}) = x_{u_j} x_{v_j}, \tag{7}$$

$$f^{(\oplus)}(x_{u_j}, x_{v_j}) = x_{u_j} + x_{v_j} - 2x_{u_j} x_{v_j}, \tag{8}$$

$$f^{(\vee)}(x_{u_j}, x_{v_j}) = x_{u_j} + x_{v_j} - x_{u_j} x_{v_j}, \tag{9}$$

with the full set of 16 relaxed operators listed in Table 1.

To learn which logic operator should be assigned to neuron j, the model associates a trainable score vector $\mathbf{w}^{(j)} = (w_0^{(j)}, \ldots, w_{15}^{(j)}) \in \mathbb{R}^{16}$ with the 16 candidate operators and converts it into a categorical distribution by a softmax:

$$p_i^{(j)} = \frac{\exp(w_i^{(j)})}{\sum_{\ell=0}^{15} \exp(w_\ell^{(j)})}, \qquad i \in \{0, \ldots, 15\}, \tag{10}$$

so that $\mathbf{p}^{(j)} = (p_0^{(j)}, \ldots, p_{15}^{(j)}) \in \Delta^{15}$. The relaxed output of neuron j is then defined as the expectation over all candidate operators,

$$z_j = \sum_{i=0}^{15} p_i^{(j)} \, f^{(i)}(x_{u_j}, x_{v_j}), \tag{11}$$

which makes the neuron differentiable and therefore trainable with gradient descent optimization.

After training, the differentiable neuron is discretized by selecting the most likely operator,

$$f_j^\star = \arg \max_{f^{(i)} \in \mathcal{F}} p_i^{(j)}, \tag{12}$$

so that inference is performed using a hard logic-gate network only. In practice, Petersen *et al.* [14] report that most neurons converge strongly to a single operator, and that the error introduced by discretization is very small, e.g., below 0.1% on the Modified National Institute of Standards and Technology (MNIST) dataset.

For classification, logic-gate networks typically use multiple output neurons per class in order to obtain graded class scores rather than a single binary decision. Let n_{out} denote the total number of output neurons and let k denote the number of classes, with n_{out}/k output neurons assigned to each class. Then the score associated with class $r \in \{0, \ldots, k-1\}$ is

$$\hat{y}_r = \sum_{j=(r \cdot n_{\text{out}}/k)+1}^{(r+1) \cdot n_{\text{out}}/k} \frac{z_j}{\tau} + \beta, \tag{13}$$

where τ is a normalization temperature and β is an optional offset [14]. After discretization, this aggregation amounts to counting the number of active output bits assigned to each class, and the predicted label is obtained from the largest class score. At this stage, inference reduces to propagating binary activations through successive two-input Boolean gates, as illustrated in Figure 1.

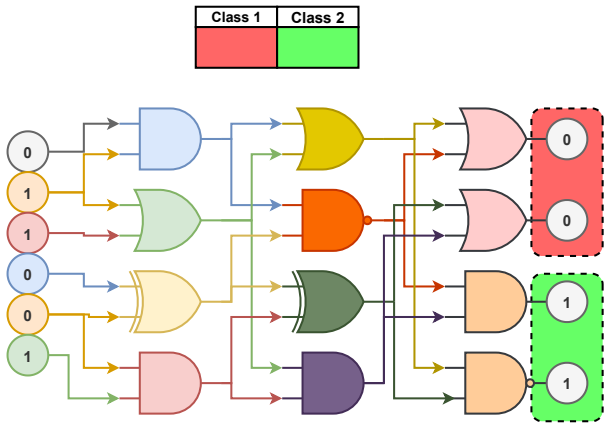

Fig. 1. Illustration of a logic-gate network in inference mode. Binary input activations are propagated through successive two-input Boolean gates, producing intermediate Boolean features across layers. The final output bits are grouped by class and can be interpreted as class-specific activations for prediction.

The resulting network from this formulation is particularly suitable for privacy-preserving inference under TFHE. Compared with arithmetic NNs, or even discretized weight-based models, this produces a representation whose computation is much more directly aligned with TFHE's Boolean execution model. Encrypted inference can therefore be viewed as the homomorphic evaluation of a learned Boolean circuit, which is precisely the setting under study in this work.

4 Proposed Framework and Threat Model

In this section, we describe the main components of the proposed EI-DDLGN and its data flow, implemented optimizations, protected assets, adversary model, and expected security strength.

4.1 Proposed Framework Overview

As shown in Fig. 2, our proposed framework, EI-DDLGN, considers a client–server inference setting for privacy-preserving deep learning. The proposed framework consists of four stages:

1. **First stage:** The client holds a private input sample m, while the server hosts a DDLGN model g which is trained in the plaintext domain using the Python library `difflogic` [14], following the differentiable relaxation described in Section 3.2. After training, each neuron is discretized by selecting one of the 16 possible two-input Boolean operators listed in Table 1. This produces a hard logic-gate network whose inference procedure consists entirely of Boolean operations. The resulting trained model g is then exported to a lightweight intermediate representation, which, in our implementation, consists of gate descriptions and metadata files. We assume that this training stage is performed by the server using data to which they have authorized access.
2. **Second Stage**: With the help of exported files from the previous stage, the server constructs the model g layer by layer using a Rust backend and instantiates a TFHE Boolean evaluation engine using `TFHE-rs` library [20]. At the client side, a `TFHE-rs` engine is also used to generate the client secret key, public evaluation server keys, and encrypt the input bits to get $\mathbf{c}$ according to (1). Then the client keeps his own key safely and sends the remaining abovementioned elements to the server. Furthermore, the public evaluation server keys enable the server to perform homomorphic gate evaluation and bootstrapping during inference.
3. **Third Stage**: The server receives $\mathbf{c}$, and the public evaluation server keys. Then it applies the homomorphic evaluation procedure to the ciphertext input $\mathbf{c}$, producing
$$\hat{\mathbf{y}}_c = \mathsf{Eval}(g, \mathbf{c}), \tag{14}$$
where $\hat{\mathbf{y}}_c$ denotes the ciphertext encoding of the prediction scores. The homomorphic deployment, Eval, is executed by propagating ciphertext bits across the logic-gate layers. The backend evaluates each layer independently and replaces the current activation vector with the ciphertext outputs of the previous layer. Since all neurons within the same layer depend only on ciphertexts from the previous layer, the evaluation can be parallelized across gates at the same layer. After deployment is done, the encrypted prediction scores $\hat{\mathbf{y}}_c$ are then sent to the client.
4. **Fourth Stage**: The client receives the encrypted prediction scores $\hat{\mathbf{y}}_c$ from the server, and then decrypts them to obtain $\hat{\mathbf{y}}$, where
$$\hat{\mathbf{y}} = g(m), \tag{15}$$
The correctness of the obtained prediction scores requires that decrypting the encrypted evaluation recovers the same result as plaintext inference, i.e.,
$$\mathsf{Dec}(\mathsf{Eval}(g, c)) = g(m). \tag{16}$$

4.2 Implemented Boolean Backend Optimizations

Beyond the basic layer-wise execution described above, our implementation of the proposed framework EI-DDLGN incorporates several practical optimizations

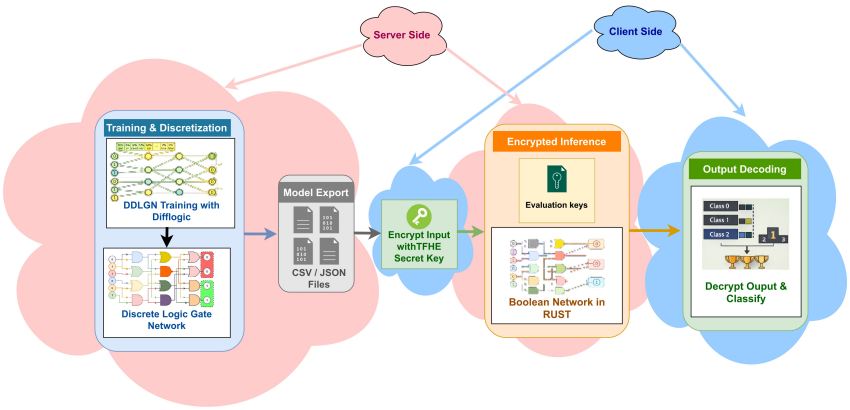

Fig. 2. Framework for privacy-preserving DDLGN inference under TFHE. The model is trained and discretized into a Boolean network, exported as CSV/JSON, and evaluated homomorphically using `TFHE.rs`. The client encrypts the input and provides evaluation keys, while the server performs encrypted inference and returns the result for local decryption and classification.

to reduce encrypted inference overhead while preserving correctness. These optimizations are applied directly at the execution level and are designed to exploit the structure of logic-gate networks under TFHE Boolean evaluation. In particular, they target redundant computations, constant-valued nodes, inexpensive unary operations, and parallelism across independent gates within a layer.

Precomputed and Reused Values. Firstly, the backend precomputes and stores two ciphertexts $\mathbf{c}_0$, $\mathbf{c}_1$ which encrypt the Boolean constants 0 and 1, respectively, and reuses them whenever a neuron corresponds to a constant-valued operator. In our Rust implementation, these constants are instantiated once through trivial encryption. This avoids repeated encryption of constant values and removes unnecessary homomorphic operations for nodes whose outputs are fixed.

Eliminated Gates. Secondly, the backend eliminates projection and identity-style gates whenever possible. Some of the 16 admissible Boolean operators correspond simply to forwarding one of the two inputs, such as

$$f_j(x_{u_j}, x_{v_j}) \in \{x_{u_j}, x_{v_j}\}. \tag{17}$$

For such cases, the backend directly returns the corresponding ciphertext without invoking a homomorphic gate operation. As a result, redundant gates are

effectively removed at evaluation time, reducing both runtime and the logical depth of the encrypted circuit.

Fast Negation Gate. Thirdly, unary negation is handled efficiently using a linear transformation on the ciphertext, $\mathsf{NOT}(c) = -c$, which does not require a programmable bootstrapping step.

Parallelization and Threading. Finally, the encrypted inference is parallelized at the layer level. Since all neurons in a fixed layer depend only on ciphertexts from the previous layer, their evaluations are independent and can be distributed across threads. This is beneficial for wider logic-gate networks, where each layer contains many independent gate evaluations.

4.3 Protected Assets

In this work, we do not aim to protect model confidentiality. The server is assumed to know the architecture and parameters of the deployed EI-DDLGN model. This assumption is appropriate for our setting, since the main objective is to protect client data during outsourced inference rather than to hide the model itself. Our primary security goal is *input privacy*: the server should learn nothing about the client input beyond what is inherently revealed by the protocol output. In particular, the server should not be able to recover the plaintext input, nor infer sensitive intermediate representations from the encrypted computation. Since inference is carried out entirely in the encrypted domain, all intermediate activations remain protected throughout evaluation.

4.4 Adversary Model

We assume an *honest-but-curious* server [13]. That is, the server follows the prescribed inference protocol correctly, but may attempt to infer information about the client's input from the encrypted data, intermediate ciphertexts, evaluation keys, or returned outputs. The client is assumed to be trusted and to keep the secret decryption key private. We further assume that the underlying TFHE scheme is semantically secure, so that ciphertexts and evaluation keys do not reveal useful information about the corresponding plaintext input. We also do not consider malicious deviations from the protocol, such as incorrect evaluation, malformed ciphertext generation, denial-of-service behavior, or side-channel leakage through timing, power, or hardware effects. Under this threat model, privacy-preserving inference reduces to securely computing $\mathsf{Eval}(g, c)$ such that the server learns nothing about m, while the client alone can decrypt the final ciphertext and recover the plaintext prediction $\hat{\mathbf{y}}$.

4.5 Security Strength

The confidentiality of the client's data, which is the main asset, is protected by the TFHE encryption. As presented in (1), the encryption strength relies on the

hardness of the LWE problem, which is parameterized by the ciphertext modulus q, the LWE vector dimension n, and the standard deviation σ_{LWE} of the natural distribution from which e is sampled .

The cryptanalysis of LWE and its variants remains an active and rigorously studied area within the cryptographic community, particularly due to its resilience against quantum computing attacks. The security level of specific LWE parameter combinations against known lattice attacks can be evaluated using tools such as the Lattice Estimator [1]. To balance cryptographic resilience, noise management across homomorphic logic gates, and practical bootstrapping latency, we adapt one of the already tested and verified parameter sets in Zama's code base [20], which is summarized in Table. 2. Evaluated against known lattice attacks using the Lattice Estimator [1], this specific set ensures a minimum of 132-bit security globally. This guarantee encompasses all ciphertext variants and the encrypted bootstrapping keys utilized throughout the protocol.

Table 2. Selected TFHE Cryptographic parameters

Parameter	n	q	σ_{LWE}	Security Level
Value	739	2^{32}	5.862×10^{-6}	132-bit

5 Complexity and Cost Analysis

In this section, we analyze the computational cost of the proposed framework EI-DDLGN. Since the trained model is discretized into a hard logic-gate network, encrypted inference reduces to the homomorphic evaluation of a layered Boolean circuit. This makes the cost structure substantially different from that of arithmetic NNs adapted to TFHE through quantization and programmable bootstrapping. In particular, for DDLGNs the dominant factor is the number and type of Boolean gate evaluations required by the discretized network, together with the layer-wise structure that determines available parallelism.

Let the discretized DDLGN contain L logic-gate layers, and let n_ℓ denote the number of neurons in layer ℓ, for $\ell = 1, \ldots, L$. Since each neuron corresponds to exactly one two-input Boolean operator, the total number of encrypted gate evaluations is

$$G = \sum_{\ell=1}^{L} n_\ell. \tag{18}$$

For a uniform-width network with n neurons per layer, this simplifies to

$$G = Ln. \tag{19}$$

Thus, ignoring key generation and input/output conversion, the encrypted inference cost grows linearly with the total number of logic gates in the network.

Proposition 1. *Let G denote the total number of logic gates in a discretized DDLGN. Under fixed TFHE backend parameters and a fixed per-gate evaluation model, the encrypted inference cost grows as*

$$T_{\text{eval}} = \Theta(G).$$

More generally, the encrypted evaluation time can be written as

$$T_{\text{eval}} = \sum_{\ell=1}^{L} \sum_{j=1}^{n_\ell} C(f_{\ell,j}), \tag{20}$$

where $C(f_{\ell,j})$ denotes the cost of evaluating the Boolean operator assigned to neuron j in layer ℓ. In the most general case, different logic operators may incur different costs. However, in our proposed framework, several important cases are effectively cheaper than generic binary gates, as we mentioned in Section 4.2. In particular, constant-valued operators which reuse precomputed encrypted constants, projection operators which simply forward an existing ciphertext, and unary negation. Therefore, the practical runtime depends not only on the total gate count G, but also on the distribution of operator types across the learned network.

Let $\mathcal{G}_{\text{const}}$, $\mathcal{G}_{\text{proj}}$, $\mathcal{G}_{\text{not}}$, and $\mathcal{G}_{\text{bin}}$ denote the sets of constant, projection, unary-negation, and general binary gates, respectively. Then a refined cost model is

$$T_{\text{eval}} \approx |\mathcal{G}_{\text{const}}|\, C_{\text{const}} + |\mathcal{G}_{\text{proj}}|\, C_{\text{proj}} + |\mathcal{G}_{\text{not}}|\, C_{\text{not}} + |\mathcal{G}_{\text{bin}}|\, C_{\text{bin}}, \tag{21}$$

where typically

$$C_{\text{const}} \approx C_{\text{proj}} \ll C_{\text{not}} \ll C_{\text{bin}}. \tag{22}$$

In practice, both constant reuse and projection elimination contribute little to the encrypted runtime, while general two-input Boolean gates dominate the evaluation cost, due to the need for PBS.

The layered structure of DDLGNs also determines the available parallelism. Since all neurons within a fixed layer depend only on ciphertexts from the previous layer, they can be evaluated independently. If P_ℓ processing units are available for layer ℓ, the parallel cost model is lower-bounded by

$$T_{\text{eval}}^{\text{par}} \approx \sum_{\ell=1}^{L} \left\lceil \frac{n_\ell}{P_\ell} \right\rceil \bar{C}_\ell, \tag{23}$$

where $\bar{C}_\ell$ is the average gate-evaluation cost in layer ℓ. Consequently, the sequential dependency is primarily across layers, not within layers. Generally, wider networks benefit more from intra-layer parallelization, whereas deeper networks increase the number of sequential evaluation stages.

The output decoding stage introduces only a small additional cost. If the network has n_{out} output neurons grouped into k classes, then the final decrypted

prediction is obtained by summing the active output bits in each class block and selecting the class with the largest vote count. The corresponding decoding complexity is

$$T_{\text{decode}} = \mathcal{O}(n_{\text{out}}), \tag{24}$$

which is negligible relative to the encrypted gate evaluation cost for the hidden logic layers. Therefore, the end-to-end inference time is dominated by

$$T_{\text{total}} = T_{\text{enc}} + T_{\text{eval}} + T_{\text{dec}}, \tag{25}$$

where T_{enc} is the time required to encrypt the input bit vector, T_{eval} is the time required to evaluate all logic-gate layers, and T_{dec} is the time required to decrypt the final output ciphertexts and recover the prediction. Moreover, it is noteworthy to mention that T_{eval} typically forms the dominant term. This is also reflected in the experimental timing measurements reported later in the paper.

This cost structure differs from QAT-based TFHE inference. In the QAT-TFHE framework [17], inference latency depends heavily on circuit bit-width, accumulator growth, and the number of programmable bootstrapping operations required to evaluate nonlinear functions over quantized integer representations. In contrast, the DDLGN setting avoids arithmetic accumulation in the hidden layers, and instead expresses inference directly as Boolean circuit execution. As a result, the complexity is more predictable and more directly tied to the learned circuit size. Thus, Boolean gate complexity emerges as the primary determinant of encrypted inference latency in our proposed framework EI-DDLGN.

6 Experimental Setup and Results

In this section, we evaluate our proposed framework EI-DDLGN described in Section 4.1 and interprets the results in light of the cost model developed in Section 5. In particular, we held the experiments to assess three aspects:

(i) The practical cost of EI-DDLGN inference.
(ii) The accuracy–latency trade-off across three EI-DDLGN model sizes.
(iii) The comparison against arithmetic-based FCNN baselines.

As we established earlier, the framework consists of training and discretizing DDLGNs in Python, exporting them as intermediate files, and evaluating the resulting Boolean circuits using a Rust backend built on TFHE.rs, with output decoding performed by class-wise vote aggregation. Moreover, the preceding complexity analysis predicts that the end-to-end cost is dominated by the encrypted evaluation term T_{eval}, and that latency should scale primarily with Boolean gate complexity.

All experiments were conducted on a desktop platform equipped with an Intel Core i9-10900 processor, 10 cores / 20 threads, a base frequency of 2.8 GHz, x86_64 architecture, and 32 GB DDR4 memory at 2933 MT/s, as summarized in Table 3. We used this hardware configuration for both the FCNN baseline measurements and the EI-DDLGN evaluation. Moreover, all the experiments are done on the MNIST dataset [6].

Table 3. Hardware specifications used for experimental evaluation.

Component	Specification
Processor	Intel Core i9-10900
CPU configuration	10 cores / 20 threads
Base frequency	2.8 GHz
Architecture	x86_64
Memory	32 GB DDR4 @ 2933 MT/s

6.1 Experiment I: End-to-End Cost Profiling of EI-DDLGN

The goal of this experiment is to quantify the practical cost of encrypted EI-DDLGN inference and to verify whether the end-to-end runtime is dominated by the encrypted evaluation term T_{eval}, as predicted by the analysis in Section 5.

We evaluate the three EI-DDLGN configurations listed in Table 4 on the MNIST dataset. The *small*, *medium*, and *large* models contain 2, 4, and 6 logic-gate layers, respectively, with 4,000, 6,000, and 8,000 neurons per layer. This corresponds to total network sizes of 8,000, 24,000, and 48,000 logic-gate neurons.

Table 4. EI-DDLGN architectures on MNIST.

Model	Layers	Neurons/layer	Total neurons	τ
EI-DDLGN (small)	2	4,000	8,000	3.0
EI-DDLGN (medium)	4	6,000	24,000	5.0
EI-DDLGN (large)	6	8,000	48,000	10.0

For each model, we report the same timing metrics introduced in Eq.(25), namely encryption time per image T_{enc}, encrypted evaluation time per image T_{eval}, decryption time per image T_{dec}, and total end-to-end time per image T_{total}.

Table 5 shows that encrypted evaluation is the dominant component of runtime for all three models. The smallest EI-DDLGN reaches an end-to-end latency of 6.995 s per image, of which 6.958 s is spent in encrypted evaluation. Similarly, the medium and large models require 20.164 s and 33.731 s per image, with T_{eval} accounting for 20.127 s and 33.633 s, respectively. This corresponds to approximately 99.5%, 99.8%, and 99.7% of the total runtime being concentrated in T_{eval}, while input encryption and output decryption remain negligible. These findings directly confirm the cost analysis from Section 5, where T_{eval} was predicted to be the dominant term.

6.2 Experiment II: Accuracy–Latency Scaling Across EI-DDLGN Model Sizes

The goal of this experiment is to study how EI-DDLGN model size affects the trade-off between predictive accuracy and encrypted inference latency.

Table 5. Accuracy and encrypted inference timing of EI-DDLGN models under TFHE Boolean evaluation on MNIST. Reported metrics include per-image encryption time T_{enc}, evaluation time T_{eval}, decryption time T_{dec}, and total end-to-end latency T_{total}.

Model	Acc. (%)	T_{enc} (s)	T_{eval} (s)	T_{dec} (s)	T_{total} (s)
EI-DDLGN (small)	92.14	0.034	6.958	0.0028	6.995
EI-DDLGN (medium)	96.11	0.033	20.127	0.0044	20.164
EI-DDLGN (large)	97.23	0.092	33.633	0.0060	33.731

We again use the three EI-DDLGN configurations in Table 4, which increase jointly in depth and width: the *small* model has 2 layers and 8,000 total neurons, the *medium* model has 4 layers and 24,000 total neurons, and the *large* model has 6 layers and 48,000 total neurons. These models are evaluated under the same TFHE Boolean backend and with the same class-decoding procedure described earlier. In addition to timing, we report their classification accuracies on MNIST.

Table 5 shows a clear trade-off between accuracy and latency. The smallest model achieves 92.14% accuracy at 6.995 s per image, while the medium model improves the accuracy to 96.11% at 20.164 s per image. The largest model further improves the accuracy to 97.23%, but increases the latency to 33.731 s per image. This trend is consistent with the expectation that larger logic-gate networks provide greater modeling capacity at the expense of more encrypted gate time evaluations. Importantly, the latency growth also supports Proposition 1. For example, moving from the *small* to the *medium* model triples the total number of neurons, from 8,000 to 24,000, while the evaluation time increases from 6.958 s to 20.127 s, which is close to linear. Moving from the *medium* to the *large* model doubles the neuron count from 24,000 to 48,000 and increases the evaluation latency from 20.127 s to 33.633 s. Although the scaling is not perfectly proportional due to implementation effects and gate-type distributions, the overall behavior remains strongly consistent with the $\Theta(G)$ dependence derived in Section 5.

6.3 Experiment III: Comparison Against Arithmetic-Based FCNN Baselines

The goal of this experiment is to compare the proposed EI-DDLGN framework against arithmetic-based TFHE-compatible FCNN baselines and assess whether the Boolean-native DDLGN formulation leads to a better accuracy–latency trade-off.

For comparison, we reproduced FCNN results from [17] under the narrow-range setting with 2-bit quantization, as summarized in Table 6. These FCNN results are averaged over 10 independent models with different random seeds. All baseline configurations use the same three-layer fully connected architecture with 192, 192, and 10 neurons. The FCNN variants differ in sparsity level, which determines the number of active connections and, indirectly, the resulting circuit bit-width and encrypted inference cost. We compare these baselines against the three EI-DDLGN models in Table 5.

Table 6. Summary of performance across different sparsity levels (narrow range, 2-bit quantization). Results are averaged over 10 independent models with different random seeds.

Model	Active conn.	Acc. (%)	Time/img (s)	Circuit bitwidth
FCNN-4	56	0.9154	88.24	6-bit: 50%, 7-bit: 50%
FCNN-6	84	0.9225	126.97	6-bit: 10%, 7-bit: 90%
FCNN-8	112	0.9245	136.68	7-bit: 100%
FCNN-10	140	0.9229	135.98	7-bit: 100%
FCNN-11	154	0.9258	132.25	7-bit: 100%
FCNN-12	168	0.9242	207.98	7-bit: 90%, 8-bit: 10%

All models follow the 3-layer FCNN architecture reported in [17]: three fully connected layers with 192, 192, and 10 neurons.

The FCNN baselines achieve accuracies between 91.51% and 92.58%, with inference times ranging from 88.24 s to 207.98 s per image depending on sparsity level and resulting circuit bit-width. By contrast, all three EI-DDLGN models are substantially faster. Even the largest EI-DDLGN requires only 33.731 s per image, which is still much lower than the fastest FCNN baseline at 88.24 s. Furthermore, the medium and large EI-DDLGN models also outperform the FCNN baselines in accuracy, reaching 96.11% and 97.23%, respectively, compared with the FCNN maximum of 92.58%. Another important observation from Table 6 is that FCNN performance remains strongly coupled to circuit bit-width: most baseline configurations operate predominantly at 7-bit width, while the highest active-connections configuration occasionally reaches 8-bit width and exhibits the highest latency. In contrast, the EI-DDLGN backend evaluates Boolean gates directly under TFHE and does not rely on quantized integer accumulation in the hidden layers. This difference leads to a more predictable and structurally aligned encrypted execution model, and explains why EI-DDLGN provides a substantially better accuracy–latency trade-off in the evaluated setting.

7 Conclusion

This paper introduced EI-DDLGN, a framework for efficient privacy-preserving inference of DDLGN under TFHE. By leveraging the Boolean-native structure of discretized DDLGNs, the proposed framework enables encrypted inference through direct homomorphic evaluation of learned logic-gate circuits. This avoids the arithmetic accumulation and circuit bit-width sensitivity that characterize QAT-based NNs inference under TFHE. The framework combines plaintext DDLGN training and discretization in Python with a TFHE Boolean backend in Rust, providing a complete pipeline from model generation to encrypted deployment.

Our analysis and experiments show that EI-DDLGN is both practical and structurally well aligned with TFHE. The complexity analysis established that encrypted inference cost scales linearly with the total number of Boolean gates, and the experimental results confirmed this trend across three EI-DDLGN model sizes. In addition, the reported timing breakdown demonstrated that the ho-

momorphic evaluation stage dominates the end-to-end latency, while encryption and decryption contribute only marginally. Compared with the reproduced QAT-FCNN baseline, EI-DDLGN achieved substantially lower encrypted inference latency while also reaching higher classification accuracy on MNIST. These findings support the central claim of this work: aligning model architecture with the underlying cryptographic computation model can significantly improve the practicality of privacy-preserving inference.

Despite these promising results, the present work has several limitations. First, the current EI-DDLGN framework is restricted to feedforward logic-gate architectures and does not yet support convolutional layers, which limits its applicability to more complex vision tasks. Second, the evaluation is conducted on MNIST and a relatively controlled experimental setting, so broader validation on more challenging datasets and tasks remains necessary. Third, our current deployment performs one-image-at-a-time encrypted inference, which does not yet exploit possible throughput gains from multi-sample evaluation. Finally, the present study focuses on input privacy under an honest-but-curious threat model and does not address model confidentiality or malicious adversaries.

These limitations motivate several directions for future work. A first priority is to extend EI-DDLGN to convolutional logic-gate architectures, enabling encrypted inference for richer image-processing pipelines while preserving the Boolean-friendly execution model. A second direction is to investigate mechanisms for classifying multiple images within a single inference procedure, which may improve throughput and better utilize the homomorphic backend. Additional future directions include evaluating EI-DDLGN on larger and more diverse datasets and refining the backend for further latency reductions.

Overall, EI-DDLGN demonstrates that DDLGNs constitute a promising model family that conserves the learning capabilities and characteristics of NNs and, at the same time, are efficient for encrypted inference under TFHE. We hope this work encourages further research at the intersection of logic-based neural architectures, homomorphic encryption, and practical privacy-preserving AI.

References

1. Albrecht, M., Bard, G.: Lattice estimator. https://github.com/malb/lattice-estimator (2021), accessed: 2023-08-22
2. Benamira, A., Guérand, T., Peyrin, T., Saha, S.: Tt-tfhe: a torus fully homomorphic encryption-friendly neural network architecture. arXiv preprint arXiv:2302.01584 (2023)
3. Bourse, F., Minelli, M., Minihold, M., Paillier, P.: Fast homomorphic evaluation of deep discretized neural networks. In: Annual International Cryptology Conference. pp. 483–512. Springer (2018)
4. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Tfhe: Fast fully homomorphic encryption over the torus: I. chillotti et al. Journal of Cryptology **33**(1), 34–91 (2020)
5. Chillotti, I., Joye, M., Paillier, P.: Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In: International Symposium on Cyber Security Cryptography and Machine Learning. pp. 1–19. Springer (2021)

6. Deng, L.: The mnist database of handwritten digit images for machine learning research [best of the web]. IEEE signal processing magazine **29**(6), 141–142 (2012)
7. Gilad-Bachrach, R., Dowlin, N., Laine, K., Lauter, K., Naehrig, M., Wernsing, J.: Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In: International conference on machine learning. pp. 201–210. PMLR (2016)
8. Huang, S., Zhou, Y., Zheng, P.: Privacy-preserving inference of binary neural network using fully homomorphic encryption. In: International Conference on Intelligent Computing. pp. 235–246. Springer (2025)
9. Ku, Y.T., Liu, F.H., Hsu, C.F., Chang, M.C., Hung, S.H., Tu, I.P., Chen, W.C.: Optimizing encrypted neural networks: Model design, quantization and fine-tuning using fhew/tfhe. Proceedings on Privacy Enhancing Technologies (2025)
10. Li, D., Chattopadhyay, A., Li, Q., Lü, Q., Wu, J., Xiang, T., Liao, X.: Cryptdnn: A fast privacy-preserving deep neural network inference architecture based on cloud-edge-client collaboration. IEEE Transactions on Network Science and Engineering (2025)
11. Li, D., Chattopadhyay, A., Lü, Q., Wu, J., Xiang, T., Liao, X.: Fsat: A faster secure convolutional neural network inference framework with adversarial training in resource-constrained scenarios. IEEE Transactions on Information Forensics and Security **21**, 798–811 (2026)
12. Lou, Q., Jiang, L.: She: A fast and accurate deep neural network for encrypted data. Advances in neural information processing systems **32** (2019)
13. Paverd, A., Martin, A., Brown, I.: Modelling and automatically analysing privacy properties for honest-but-curious adversaries. Tech. Rep (2014)
14. Petersen, F., Borgelt, C., Kuehne, H., Deussen, O.: Deep differentiable logic gate networks. Advances in Neural Information Processing Systems **35**, 2006–2018 (2022)
15. Roy, A., Roy, K.: Dct-cryptonets: Scaling private inference in the frequency domain. arXiv preprint arXiv:2408.15231 (2024)
16. Selvakumar, S., Senthilkumar, B.: A privacy preserving machine learning framework for medical image analysis using quantized fully connected neural networks with tfhe based inference: S. selvakumar, s. b. Scientific Reports **15**(1), 27880 (2025)
17. Stoian, A., Frery, J., Bredehoft, R., Montero, L., Kherfallah, C., Chevallier-Mames, B.: Deep neural networks for encrypted inference with tfhe. In: International Symposium on Cyber Security, Cryptology, and Machine Learning. pp. 493–500. Springer (2023)
18. Yassin, M.Y., Nasir, A.A., Taha, M.: Novel dual-domain chaotic image cryptosystem for cybersecurity applications. IEEE Access **12**, 123694–123715 (2024)
19. Yassin, M.Y., Nasir, A.A., Taha, M., Iqbal, N.: Enhanced dft-based chaotic image block cipher with several modes of operation. In: 2023 Twelfth International Conference on Image Processing Theory, Tools and Applications (IPTA). pp. 1–6. IEEE (2023)
20. Zama: TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data (2022), https://github.com/zama-ai/tfhe-rs